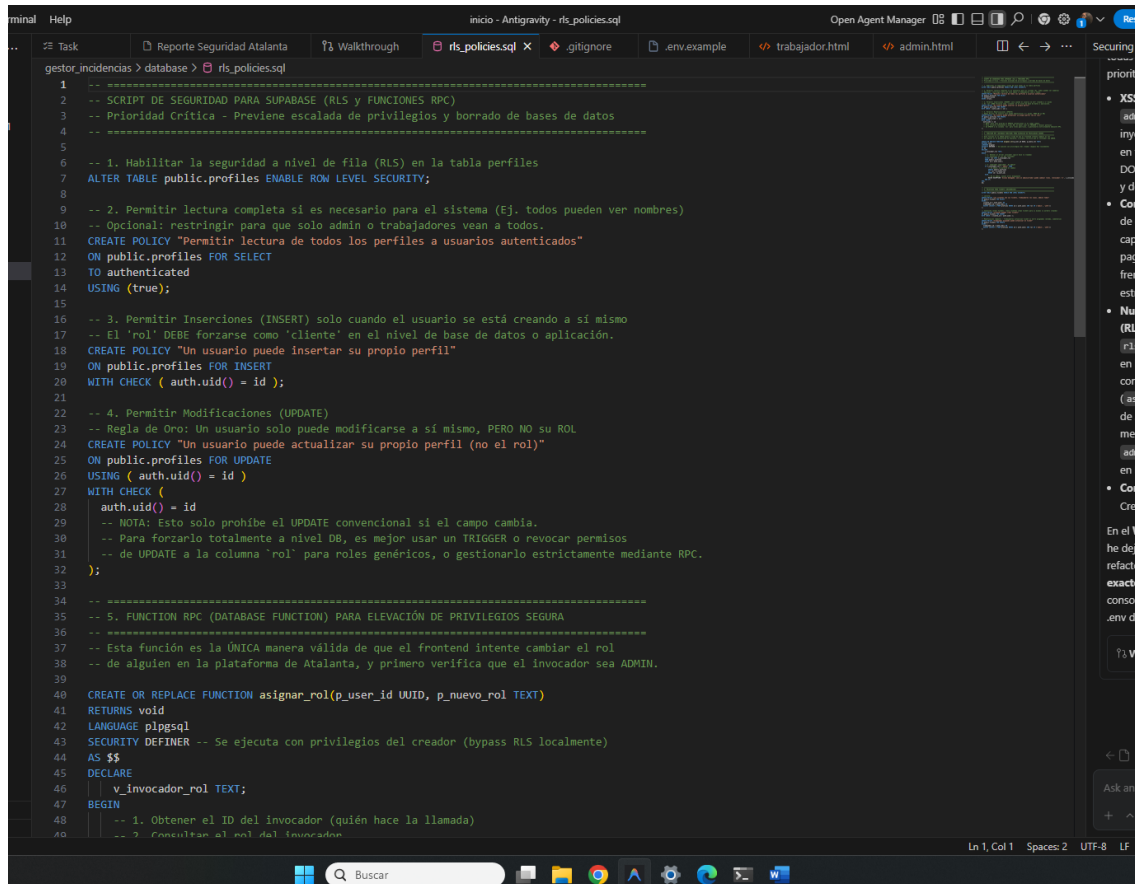


Todo lo que agregamos conforme a seguridad 24/03/2026



```
1  -- =====
2  -- SCRIPT DE SEGURIDAD PARA SUPABASE (RLS y FUNCIONES RPC)
3  -- Prioridad Crítica - Previene escalada de privilegios y borrado de bases de datos
4  -- =====
5
6  -- 1. Habilitar la seguridad a nivel de fila (RLS) en la tabla perfiles
7  ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;
8
9  -- 2. Permitir lectura completa si es necesario para el sistema (Ej. todos pueden ver nombres)
10 -- Opcional: restringir para que solo admin o trabajadores vean a todos.
11 CREATE POLICY "Permitir lectura de todos los perfiles a usuarios autenticados"
12 ON public.profiles FOR SELECT
13 TO authenticated
14 USING (true);
15
16 -- 3. Permitir Inserciones (INSERT) solo cuando el usuario se está creando a sí mismo
17 -- El 'rol' DEBE forzarle como 'cliente' en el nivel de base de datos o aplicación.
18 CREATE POLICY "Un usuario puede insertar su propio perfil"
19 ON public.profiles FOR INSERT
20 WITH CHECK ( auth.uid() = id );
21
22 -- 4. Permitir Modificaciones (UPDATE)
23 -- Regla de Oro: Un usuario solo puede modificarse a sí mismo, PERO NO su ROL
24 CREATE POLICY "Un usuario puede actualizar su propio perfil (no el rol)"
25 ON public.profiles FOR UPDATE
26 USING ( auth.uid() = id )
27 WITH CHECK (
28     auth.uid() = id
29     -- NOTA: Esto solo prohíbe el UPDATE convencional si el campo cambia.
30     -- Para forzarlo totalmente a nivel DB, es mejor usar un TRIGGER o revocar permisos
31     -- de UPDATE a la columna 'rol' para roles genéricos, o gestionarlo estrictamente mediante RPC.
32 );
33
34 -- =====
35 -- 5. FUNCTION RPC (DATABASE FUNCTION) PARA ELEVACIÓN DE PRIVILEGIOS SEGURA
36 -- =====
37 -- Esta función es la ÚNICA manera válida de que el frontend intente cambiar el rol
38 -- de alguien en la plataforma de Atalanta, y primero verifica que el invocador sea ADMIN.
39
40 CREATE OR REPLACE FUNCTION asignar_rol(p_user_id UUID, p_nuevo_rol TEXT)
41 RETURNS void
42 LANGUAGE plpgsql
43 SECURITY DEFINER -- Se ejecuta con privilegios del creador (bypass RLS localmente)
44 AS $$
45 DECLARE
46     v_invocador_rol TEXT;
47 BEGIN
48     -- 1. Obtener el ID del invocador (quién hace la llamada)
49     -- 2. Consultar el rol del invocador.
```

```

-- =====
-- SCRIPT DE SEGURIDAD PARA SUPABASE (RLS y FUNCIONES RPC)
-- Prioridad Crítica - Previene escalada de privilegios y borrado de bases de datos
-- =====

-- 1. Habilitar la seguridad a nivel de fila (RLS) en la tabla perfiles
ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;

-- 2. Permitir lectura completa si es necesario para el sistema (Ej. todos pueden ver nombres)
-- Opcional: restringir para que solo admin o trabajadores vean a todos.
CREATE POLICY "Permitir lectura de todos los perfiles a usuarios autenticados"
ON public.profiles FOR SELECT
TO authenticated
USING (true);

-- 3. Permitir Inserciones (INSERT) solo cuando el usuario se está creando a sí mismo
-- El 'rol' DEBE forzarse como 'cliente' en el nivel de base de datos o aplicación.
CREATE POLICY "Un usuario puede insertar su propio perfil"
ON public.profiles FOR INSERT
WITH CHECK ( auth.uid() = id );

-- 4. Permitir Modificaciones (UPDATE)
-- Regla de Oro: Un usuario solo puede modificarse a sí mismo, PERO NO su ROL
CREATE POLICY "Un usuario puede actualizar su propio perfil (no el rol)"
ON public.profiles FOR UPDATE
USING ( auth.uid() = id )
WITH CHECK (
    auth.uid() = id
    -- NOTA: Esto solo prohíbe el UPDATE convencional si el campo cambia.
    -- Para forzarlo totalmente a nivel DB, es mejor usar un TRIGGER o revocar permisos
    -- de UPDATE a la columna `rol` para roles genéricos, o gestionarlo estrictamente mediante RPC.
);

-- =====
-- 5. FUNCTION RPC (DATABASE FUNCTION) PARA ELEVACIÓN DE PRIVILEGIOS SEGURA
-- =====
-- Esta función es la ÚNICA manera válida de que el frontend intente cambiar el rol
-- de alguien en la plataforma de Atalanta, y primero verifica que el invocador sea ADMIN.

CREATE OR REPLACE FUNCTION asignar_rol(p_user_id UUID, p_nuevo_rol TEXT)
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER -- Se ejecuta con privilegios del creador (bypass RLS localmente)
AS $$
DECLARE
    v_invocador_rol TEXT;
BEGIN
    -- 1. Obtener el ID del invocador (quién hace la llamada)
    -- 2. Consultar el rol del invocador

```

gestor_incidencias > database >  ris_policies.sql

```
38 -- de alguien en la plataforma de Atalanta, y primero verifica que el invocador sea ADMIN.
39
40 CREATE OR REPLACE FUNCTION asignar_rol(p_user_id UUID, p_nuevo_rol TEXT)
41 RETURNS void
42 LANGUAGE plpgsql
43 SECURITY DEFINER -- Se ejecuta con privilegios del creador (bypass RLS localmente)
44 AS $$
45 DECLARE
46     v_invocador_rol TEXT;
47 BEGIN
48     -- 1. Obtener el ID del invocador (quién hace la llamada)
49     -- 2. Consultar el rol del invocador
50     SELECT rol INTO v_invocador_rol
51     FROM public.profiles
52     WHERE id = auth.uid();
53
54     -- 3. Comprobar seguridad: ¿Es Admin?
55     IF v_invocador_rol = 'admin' THEN
56         -- Es admin, se permite el cambio
57         UPDATE public.profiles
58         SET rol = p_nuevo_rol
59         WHERE id = p_user_id;
60     ELSE
61         -- No es admin, lanzar error bloqueante
62         RAISE EXCEPTION 'Acceso denegado: solo un administrador puede cambiar roles. (Invocador: %)', v_invocador_rol;
63     END IF;
64 END;
65 $$;
66
67
68 -- =====
69 -- 6. POLÍTICAS PARA TICKETS (INCIDENCIAS)
70 -- =====
71 ALTER TABLE public.tickets ENABLE ROW LEVEL SECURITY;
72
73 -- Lectura:
74 CREATE POLICY "Los clientes ven sus tickets, trabajadores los suyos, admins todos"
75 ON public.tickets FOR SELECT
76 USING (
77     cliente_id = auth.uid() OR
78     trabajador_id = auth.uid() OR
79     EXISTS (SELECT 1 FROM profiles WHERE id = auth.uid() AND rol IN ('admin', 'jefe'))
80 );
81
82 -- Inserción (Crear Ticket) - Solo clientes crean tickets para si mismos (o workers creando)
83 CREATE POLICY "Clientes pueden crear tickets"
84 ON public.tickets FOR INSERT
85 WITH CHECK ( cliente_id = auth.uid() );
86
```

```

55     IF v_invocador_rol = 'admin' THEN
56         SET rol = p_nuevo_rol
57         WHERE id = p_user_id;
58     ELSE
59         -- No es admin, lanzar error bloqueante
60         RAISE EXCEPTION 'Acceso denegado: solo un administrador puede cambiar roles. (Invocador: %)', v_invocador_rol;
61     END IF;
62 END;
63 $$$;
64
65 -- =====
66 -- 6. POLÍTICAS PARA TICKETS (INCIDENCIAS)
67 -- =====
68 ALTER TABLE public.tickets ENABLE ROW LEVEL SECURITY;
69
70 -- Lectura:
71 CREATE POLICY "Los clientes ven sus tickets, trabajadores los suyos, admins todos"
72 ON public.tickets FOR SELECT
73 USING (
74     cliente_id = auth.uid() OR
75     trabajador_id = auth.uid() OR
76     EXISTS (SELECT 1 FROM profiles WHERE id = auth.uid() AND rol IN ('admin', 'jefe'))
77 );
78
79 -- Inserción (Crear Ticket) - Solo clientes crean tickets para sí mismos (o workers creando)
80 CREATE POLICY "Clientes pueden crear tickets"
81 ON public.tickets FOR INSERT
82 WITH CHECK ( cliente_id = auth.uid() );
83
84 -- Modificación (Update) - Trabajadores actualizan ticket si están asignados (estado, comentario)
85 CREATE POLICY "Trabajador asignado puede actualizar el ticket"
86 ON public.tickets FOR UPDATE
87 USING (
88     trabajador_id = auth.uid() OR
89     EXISTS (SELECT 1 FROM profiles WHERE id = auth.uid() AND rol IN ('admin', 'jefe'))
90 );
91
92
93
94

```

inal

Help

inicio - Antigravity - .gitignore

Task

Reporte Seguridad Atalanta

Walkthrough

rls_policies.sql

.gitignore X

gestor_incidencias > .gitignore

1

Node modules

2

node_modules/

3

4

Entorno y Secretos

5

.env

6

.env.local

7

.env.production

8

.env.development

9

10

Logs

11

npm-debug.log*

12

yarn-debug.log*

13

yarn-error.log*

14

*.log

15

16

SO

17

.DS_Store

18

Thumbs.db

19


```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1.0">
6 <title>Atalanta | Panel Trabajador</title>
7 <link rel="stylesheet" href="dashboard.css">
8 </head>
9 <body>
10
11 <aside class="sidebar">
12 <div class="sidebar-logo">
13 
14 <span class="sidebar-logo-text">ATALANTA</span>
15 </div>
16 <span class="sidebar-role-badge">ROL: TRABAJADOR</span>
17
18 <span class="nav-section-title">Menú</span>
19 <a class="nav-item active" onclick="showSection('tickets')">
20 <span class="icon">📌</span> Mis asignaciones
21 </a>
22 <a class="nav-item" onclick="showSection('perfil')">
23 <span class="icon">👤</span> Mi Perfil
24 </a>
25
26 <div class="sidebar-footer">
27 <div style="font-family: 'Share Tech Mono', monospace; font-size: .6rem; color: #212529; margin-bottom: .6rem;" id="user-email">
28 <button class="btn-logout" onclick="logout()"> CERRAR SESIÓN</button>
29 </div>
30 </div>
31
32 <main class="main">
33
34 <section id="section-tickets">
35 <div class="page-header">
36 <h1>Mis Asignaciones</h1>
37 <p>Tickets asignados a ti - actualiza el estado y añade comentarios</p>
38 </div>
39
40 <div class="cards-grid">
41 <div class="stat-card"><div class="stat-label">Asignados</div><div class="stat-value" id="stat-total"></div></div>
42 <div class="stat-card"><div class="stat-label">En progreso</div><div class="stat-value" id="stat-progreso"></div></div>
43 <div class="stat-card"><div class="stat-label">Resueltos</div><div class="stat-value" id="stat-resueltos"></div></div>
44 </div>
45
46 <div class="table-section">
47 <div class="table-header"><h2>Tickets asignados</h2></div>
48 <table>
49 <thead>
```

INFORME GENERAL

```
Informe de Implementación de Mitigaciones de Seguridad

1. Eliminación de XSS
Se implementó la función estricta escapeHTML en ambos dashboards (trabajador.html y admin.html) para codificar todas las variables de los usuarios (nombre, apellido, email, contenido de comentarios) antes de inyectarlos en el DOM vía <%=escapeHTML%>. Adicionalmente, se escaparon comillas dobles y simples al construir propiedades dinámicas HTML como onclick. Por qué funciona: Al escapar caracteres especiales como < y >, obligamos al navegador a interpretar los payloads maliciosos como simple texto, neutralizando cualquier ejecución de scripts inyectados en la base de datos (Stored XSS).

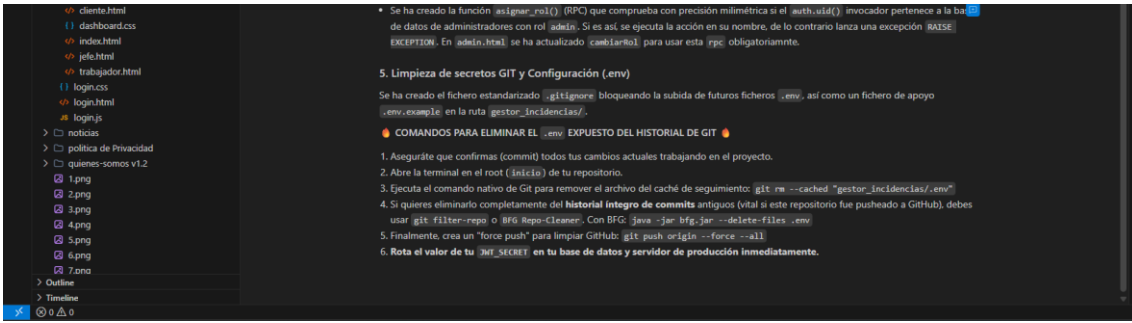
2. Paginación y Optimización de Consultas (Anti-Fuga de Datos)
Se reemplazaron los <select*> en los paneles por selecciones estrictas como <select('id, nombre, apellido, email, rol, created_at')>. Además, se agregó encadenamiento <range(0, 9)> para implementar una paginación que limita la transferencia de objetos e información a sólo la necesaria para la vista, mitigando riesgos de raspado rápido de la base de datos y optimizando carga.

3. Manejo Correcto de Errores
Se añadieron validaciones defensivas usando <console.error> para errores de base de datos o sesión en el flujo principal (admin.html, trabajador.html). Ya no se muestran datos de sesión en crudo, garantizando privacidad en entornos de producción.

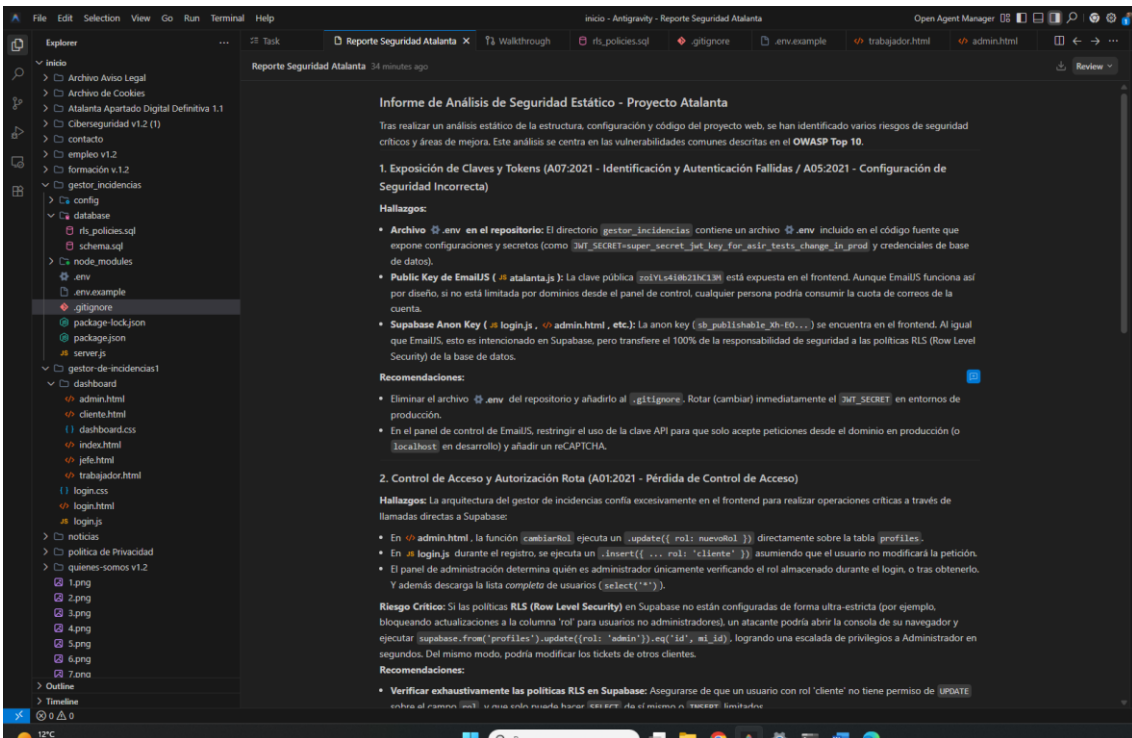
4. Blindaje de Supabase (RLS y Database RPC)
La lógica de validación de roles de administrador se ha transferido del cliente (HTML) a la Base de Datos.
• Se ha generado un script SQL de configuración (rls_policies.sql) guardado en el archivo del proyecto gestor_incidentes/database/. Este script crea restricciones severas RLS para las tablas profiles y tickets. Un usuario con rol cliente ahora no puede actualizar bajo ningún concepto su columna rol.
• Se ha creado la función assign_role() (RPC) que comprueba con precisión milimétrica si el auth.uid() invocador pertenece a la base de datos de administradores con rol admin. Si es así, se ejecuta la acción en su nombre, de lo contrario lanza una excepción RAISE EXCEPTION. En admin.html se ha actualizado cambiarRol para usar esta rpc obligatoriamente.

5. Limpieza de secretos GIT y Configuración (.env)
Se ha creado el fichero estandarizado .gitignore bloqueando la subida de futuros ficheros .env, así como un fichero de apoyo .env.example en la ruta gestor_incidentes/.

COMANDOS PARA ELIMINAR EL .env EXPUESTO DEL HISTORIAL DE GIT
1. Asegúrate que confirmas (commit) todos tus cambios actuales trabajando en el proyecto.
2. Abre la terminal en el root (inicio) de tu repositorio.
3. Ejecuta el comando nativo de Git para remover el archivo del caché de seguimiento: git rm --cached "gestor_incidentes/.env"
4. Si quieres eliminarlo completamente del historial íntegro de commits antiguos (vital si este repositorio fue pushado a GitHub), debes usar git filter-repo o BFG Repo-Cleaner. Con BFG: java -jar bfg.jar --delete-files .env
5. Finalmente, crea un "force push" para limpiar GitHub: git push origin --force --all
```



INFORME DE DETECCIONES DE ERRORES



FileEditSelectionViewGoRunTerminalHelp

Reporte Seguridad Atalanta

Open Agent Manager

Inicio

Archivo Aviso Legal

Archivo de Cookies

Atalanta Apartado Digital Definitiva 1.1

Ciberseguridad v1.2 (1)

contacto

empleo v1.2

formación v1.2

gestor_incidentes

config

database

rls_policies.sql

schema.sql

node_modules

env

env.example

gitignore

package-lock.json

package.json

server.js

gestor-de-incidencias1

dashboard

admin.html

cliente.html

dashboard.css

index.html

jeffo.html

trabajador.html

login.css

login.html

login.js

noticias

politica de Privacidad

quienes-somos v1.2

1.png

2.png

3.png

4.png

5.png

6.png

7.png

Outline

Timeline

Reporte Seguridad Atalanta

34 minutes ago

- En el panel de control de tmaius, restringir el uso de la clave API para que solo acepte peticiones desde el dominio en produccion (o localhost en desarrollo) y añadir un reCAPTCHA.

2. Control de Acceso y Autorización Rota (A01-2021 - Pérdida de Control de Acceso)

Hallazgos: La arquitectura del gestor de incidencias confía excesivamente en el frontend para realizar operaciones críticas a través de llamadas directas a Supabase:

- En admin.html, la función cambiarRol ejecuta un .update({ rol: nuevoRol }) directamente sobre la tabla profiles.
- En login.js durante el registro, se ejecuta un .insert({ ... rol: 'cliente' }) asumiendo que el usuario no modificará la petición.
- El panel de administración determina quién es administrador únicamente verificando el rol almacenado durante el login, o tras obtenerlo. Y además descarga la lista completa de usuarios (select(**)).

Riesgo Crítico: Si las políticas RLS (Row Level Security) en Supabase no están configuradas de forma ultra-estricta (por ejemplo, bloqueando actualizaciones a la columna 'rol' para usuarios no administradores), un atacante podría abrir la consola de su navegador y ejecutar supabase.from('profiles').update({rol: 'admin'}).eq('id', mi_id), logrando una escalada de privilegios a Administrador en segundos. Del mismo modo, podría modificar los tickets de otros clientes.

Recomendaciones:

- Verificar exhaustivamente las políticas RLS en Supabase: Asegurarse de que un usuario con rol 'cliente' no tiene permiso de UPDATE sobre el campo rol, y que solo puede hacer SELECT de sí mismo o INSERT limitados.
- Traspassar la lógica de cambios de credenciales y roles críticos a un entorno de backend seguro (p. ej. Edge Functions en Supabase o utilizando el backend de express en gestor_incidentes) que se ejecute con privilegios de administrador (Service Role Key) tras verificar exhaustivamente quién hace la petición.

3. Falta de Sanitización de Datos y Riesgo de XSS (A03:2021 - Inyección)

Hallazgos: En los archivos de los dashboards (como admin.html) los datos devueltos por la base de datos se inyectan directamente en el DOM utilizando innerHTML sin ningún tipo de escape o sanitización. Ejemplo de admin.html:

```
javascript

tbody.innerHTML += `
<tr>
<td>${u.nombre} ${u.apellido}</td>
...
</tr>`;


```

Riesgo Crítico (Stored XSS): Si un atacante se registra (paso 2 de login.js) ingresando como su nombre el valor o un script malicioso para robar sesiones, este código malicioso se guardará en la base de datos. Cuando el Administrador inicie sesión y visualice la lista de usuarios, el navegador ejecutará el script del atacante, otorgando al atacante acceso al panel con los mismos permisos que el administrador (y capacidad de robar la sesión completa).

Recomendaciones:

- Reemplazar el uso manual de innerHTML por métodos seguros como textContent o utilizar algún framework moderno que sanitize

File

Edit

Selection

View

Go

Run

Terminal

Help

📁

Explorer

...

🔍

🔗

🔧

🖨

📦

▼ inicio

> 📁 Archivo Aviso Legal

> 📁 Archivo de Cookies

> 📁 Atalanta Apartado Digital Definitiva 1.1

> 📁 Ciberseguridad v1.2 (1)

> 📁 contacto

> 📁 empleo v1.2

> 📁 formación v1.2

▼ 📁 gestor_incidentes

> 📁 config

▼ 📁 database

> 📄 rls_policies.sql

> 📄 schema.sql

> 📁 node_modules

> ⚙ .env

> 📄 .env.example

> 🔴 .gitignore

> 📄 package-lock.json

> 📄 package.json

> 📄 server.js

▼ 📁 gestor-de-incidencias1

▼ 📁 dashboard

> </> admin.html

> </> cliente.html

> {} dashboard.css

> </> index.html

> </> jefe.html

> </> trabajador.html

> {} login.css

> </> login.html

> 📄 login.js

> 📁 noticias

> 📁 politica de Privacidad

> 📁 quienes-somos v1.2

> 🖼 1.png

> 🖼 2.png

> 🖼 3.png

> 🖼 4.png

> 🖼 5.png

> 🖼 6.png

> 🖼 7.png

> Outline

> Timeline

Task

Reporte Seguridad Atalanta

Reporte Seguridad Atalanta 34 minutos ago

3. Falta de Sanitización

Hallazgos: En los archivos se encuentra el DOM utilizando `innerHTML` para insertar datos sin sanitizarlos.

```
javascript
tbody.innerHTML +=
  <tr>
    <td>${u.nombre}
    ...
  </tr>`;
```

Riesgo Crítico (Stored XSS): El administrador puede iniciar sesión en el panel con los mismos datos por defecto.

Recomendaciones:

- Reemplazar el uso de `innerHTML` por `document.createElement` para crear elementos HTML de forma segura.
- Si se requiere construir HTML dinámicamente, se debe escapar los caracteres especiales en la cadena de texto.

4. Configuraciones de Seguridad

Hallazgos:

- En el backend Node.js se utiliza `process.env` para almacenar credenciales de la base de datos (Supabase).
- No hay validación de contraseñas u obligatorias.

Recomendaciones:

- Evaluar si se utilizará la base de datos de Supabase o si se creará una propia.
- Definir límites estrictos de contraseñas, como longitud mínima, máxima, dispareja, etc.

5. Diseño Inseguro

Hallazgos:

- En `admin.html` se utiliza `document.location` para redirigir al usuario al iniciar sesión.

File

Edit

Selection

View

Go

Run

Terminal

Help

Reporte Seguridad Atalanta

X

Reporte Seguridad Atalanta

34 minutos ago

- Reemplazar el uso n
datos por defecto.
- Si se requiere const
`${u.nombre}` en la

4. Configuraciones

Hallazgos:

- En el backend Node
''). Además, el bac
Supabase).
- No hay validación se
contraseñas u oblig

Recomendaciones:

- Evaluar si se utilizara
Mezclar ambas pue
- Definir límites estric
máxima, disparador

5. Diseño Inseguro

Hallazgos:

- En `</> admin.html` ,
- Riesgo:** Si un usuari
datos de usuarios (e

Recomendaciones:

- Evitar `SELECT *` y li
- Implementar pagina

Resumen de Priorid

- [CRÍTICO]** Reparar e
Almacenado.
- [CRÍTICO]** Revisar y
propios roles o alter
- [ALTA]** Eliminar el a
producción.
- [MEDIA]** Configurar

Explorer

...

inicio

Archivo Aviso Legal

Archivo de Cookies

Atalanta Apartado Digital Definitiva 1.1

Ciberseguridad v1.2 (1)

contacto

empleo v1.2

formación v1.2

gestor_incidentes

config

database

rls_policies.sql

schema.sql

node_modules

.env

.env.example

.gitignore

package-lock.json

package.json

server.js

gestor-de-incidentes1

dashboard

admin.html

cliente.html

dashboard.css

index.html

jefe.html

trabajador.html

login.css

login.html

login.js

noticias

politica de Privacidad

quienes-somos v1.2

1.png

2.png

3.png

4.png

5.png

6.png

7.png

Outline

Timeline

Task

0

0